



Assessments

Chapter 1

1. No. The Windows App SDK is supported only by the new .NET runtime, so you must migrate your Windows Forms application to .NET 5.0 or .NET 6.0 first.
2. MSIX is the best deployment technology, not only for applications that integrate the Windows App SDK, but also for every Windows application. It delivers a clean install and uninstall experience, built-in features such as automatic updates, and network bandwidth optimization. However, if, for any reason, you can't adopt the MSIX technology, the Windows App SDK also supports the opportunity to work unpackaged, which means that you can deploy your apps with any existing technology.
3. No. The Windows App SDK includes other features, such as resource management, push notifications, and activations.

Chapter 2

1. The best approach is to implement the **INotifyPropertyChanged** interface, which gives you access to an event that you must raise every time a property value changes. Another supported approach is to use dependency properties, but they are harder to declare and maintain, so they should be used only when you're working with XAML controls (such as user control).
2. False. The goal of using styles is to maximize the re-use of resources, so they should be declared at the highest possible level,

such as the **Page** container or the entire application. Even better, you can use resource dictionaries to centralize styles in a single place and organize them in a more structured way.

3. False. The default binding mode is **OneWay**, which means that only the target will receive notifications when the source changes and not the other way around. If you need to support two-way communication, you must set the binding mode to **TwoWay**.

Chapter 3

1. False. Thanks to the function support offered by **x:Bind**, you can also use system functions (such as **String.Format()**) directly in a markup expression.
2. False. With the help of the Windows Community Toolkit, you can introduce an extension method called **EnqueueAsync()** that supports the **async/await** pattern, enabling you to wait until the dispatched action is completed before performing additional tasks.
3. False. The Windows App SDK will take care of loading the proper resource file, based on the user configuration.

Chapter 4

1. False. It was true for UWP apps but, since Windows App SDK apps are based on the Win32 model, they have more flexibility. They will be automatically terminated only if the application raises an exception that isn't handled.
2. False. If packaged apps can support all the activation contracts offered by the UWP, at the time of writing, unpackaged apps can support only four types of activation: launch, file, protocol, and start up task.
3. True. This is the opposite behavior of UWP apps which, instead, are single-instance by default.

Chapter 5

1. False. **Canvas** uses absolute positioning to place the controls on a page, making it hard to react to changes in the size of the window. It's better to use controls such as **Grid** or **RelativePanel**, which better support layouts that can dynamically change.
2. False. The **NavigationView** control has a built-in **VisualStateManager** which takes care of changing its display mode (Full, Compact, or Minimized) automatically based on the available space.
3. It's called Visual Layer and it sits between the XAML layer and the DirectX one.

Chapter 6

1. False. Libraries such as MVVM Toolkit or Prism can help you implement the pattern in a faster and more reliable way, but it isn't an absolute requirement. You can choose to build the required building blocks on your own since the pattern is based on standard C# and XAML features.
2. One of the goals of the MVVM pattern is to decouple as much as possible all the components, so that you evolve, maintain, and test them independently. Thanks to messages, you can exchange data between two classes without coupling them.
3. Ideally, the answer should be false. The goal of the Model domain is to be platform-agnostic. By introducing the **INotifyPropertyChanged** implementation in classes that belong to the Model layer, you create a dependency with XAML-based technologies, making it hard to reuse the same classes in a different type of project, such as a web application or a REST API.

Chapter 7

1. False. Even though they are both based on XAML, and you can re-use most of the knowledge, they support different features, a different set of controls, and different namespaces. As such, some work is needed to migrate your UI layer.
2. False. The **x:Bind** markup expression uses the code-behind to resolve the binding channels. As such, you must expose your ViewModel through a property in the code-behind class.
3. True. Windows App SDK applications, being based on the Win32 ecosystem, have a simpler life cycle, so they support only a single activation point: the **OnLaunched** event of the **App** class. Multiple activation points are supported, but you must manage them directly in this event.

Chapter 8

1. False. The WebView2 control requires the WebView2 runtime, which is a special version of the Edge engine optimized for hybrid scenarios.
2. False. You just need to set the **TargetFramework** property of your project to use one of the available workloads for Windows 10 and the library will be installed automatically.
3. True. Only the use of local storage is supported just by packaged apps, but everything else also works with unpackaged apps, developed with any .NET UI platform.

Chapter 9

1. False. They are also supported by WPF and Windows Forms applications. However, based on the platform of your choice, you must place the code to handle the activation in a different location, since every platform has its own way of handling the startup phase.
2. False. It's required when you want to replace existing notifications, rather than adding a new one; or when you need to retrieve it at a

later point in time, for example, to delete it. However, notifications are also displayed just fine without a tag.

3. False. A packaged application must register two extensions in the manifest so that Windows can properly manage the activation phase.

Chapter 10

1. ONNX is an open source standard to store an ML model in a file, which can be exchanged and consumed by different types of applications. Being portable, it's a good fit for Edge scenarios, where the model must be evaluated locally.
2. False. Thanks to WinML, an ML model can be evaluated on the local machine, without needing any internet connection and any communication with a cloud service.
3. Custom Vision is a service provided by Azure that removes the complexity of learning ML algorithms to train a model to recognize objects, sentiments, and so on. You just need to provide a series of images as input, classify them, and train the model. Custom Vision is a great companion for WinML since you can export the trained model to ONNX.

Chapter 11

1. False. Windows Package Manager is a tool that makes it more convenient to install applications from existing sources, such as the Microsoft Store or a website. The repository only contains manifests that describe the application and the URL from which to grab the installer, but it doesn't host the actual application.
2. True. If you want granular control over the update process, you can use a set of APIs to manage updates within your application. However, you aren't required to use them. You can just rely on the update settings you can define in the App Installer file.

3. False. The Microsoft Store is the perfect distribution platform for consumer apps since it's already installed in Windows and greatly simplifies the installation experiences for users. However, most of its features target a consumer audience, such as the requirement to use a personal Microsoft account to purchase and download applications. If you want to create a personal catalog of applications for your company (which can also include applications from the Microsoft Store), soon Windows Package Manager will enable you to host your own private repository.

Chapter 12

1. False. The easiest option is to install the missing requirements in the first steps in a workflow (such as a specific version of .NET or Java). If this isn't an option for you (for example, you need a tool that can't be installed in an unattended way), you can create your own self-runner and connect it to GitHub. This way, you can continue to use the powerful features offered by GitHub Actions, but the workflow will be executed on one of your machines.
2. False. Even if all the modern CI/CD platforms are using YAML, each of them has its own different syntax and actions available.
3. True, even if MSIX is the simplest one since it doesn't require any additional tool. You just need Visual Studio and the Windows 10 SDK, which is pre-installed on all the Windows build machines provided by the most important vendors.